CSC3002 — Streams Summary

> **Core idea:** A **stream** is an object that represents a flow of data between a **source** and a **destination**.

```
Source -> Input Stream -> Program/Variable
Program/Variable -> Output Stream -> Destination
```

Examples of sources/destinations: keyboard, console, file, and string.

---

1. Standard C++ Streams

Include:

```
#include <iostream>
using namespace std;
```

`cout` — Standard Output Stream

`cout` sends data from the program to the console.

```
int total = 10;
cout << "The total is " << total << endl;
```

- `<<` = **insertion operator**
- `endl` = move to the next line
- Multiple `<<` operations can be chained.

`cin` — Standard Input Stream

`cin` reads data from the keyboard into variables.

```
int limit;
cout << "Enter exponent limit: ";
cin >> limit;
```

- `>>` = **extraction operator**
- Multiple values can also be read in one chain:

```
int a, b;
cin >> a >> b;
```

---

2. Formatted Output

For C++-style formatted output:

```
#include <iomanip>
```

Common manipulators:

Manipulator	Meaning
`endl`	Move to next line
`setw(n)`	Set width of the **next** value to `n` characters

Manipulator	Meaning
`setprecision(n)`	Control number of displayed digits
`fixed`	Fixed-point notation
`scientific`	Scientific notation
`left`	Left alignment
`right`	Right alignment
`setfill(ch)`	Character used for padding
`showpos` / `noshowpos`	Show/hide `+` sign
`showpoint` / `noshowpoint`	Control decimal point display
`boolalpha` / `noboolalpha`	Display bool as `true/false` or `1/0`
`uppercase` / `nouppercase`	Control uppercase formatting

Example:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double x = 3.1415926535;

    cout << fixed << setprecision(2) << x << endl;
    // 3.14
}
```

Important: `setprecision`

Without `fixed` / `scientific`, `setprecision(n)` controls **significant digits**.

With `fixed` or `scientific`, it controls the number of digits **after the decimal point**.

---

3. C-Style Formatted Output/Input

C++ can also use functions from:

```
#include <cstdio>
```

Example:

```
char c = 'a';
int i = 1;
double d = 3.14159;

printf("%c\n", c);
printf("%d\n", i);
printf("%.2f\n", d);
```

Common format specifiers:

Specifier	Meaning
`%d`, `%i`	Signed decimal integer
`%u`	Unsigned decimal integer
`%o`	Octal
`%x`, `%X`	Hexadecimal

Specifier	Meaning
`%f`	Floating point
`%e` , `%E`	Scientific notation
`%c`	Character
`%s`	String
`%p`	Pointer address
`%%`	Print `%`

In C++, the lecture encourages using `cout <<` instead of `printf` , and `cin >>` instead of `scanf` , while still knowing the C-style interface.

---

4. Input Whitespace Manipulators

Normally, `cin >>` skips whitespace such as spaces and newlines.

Useful manipulators:

Manipulator	Meaning
`skipws`	Skip whitespace
`noskipws`	Do **not** skip whitespace
`ws`	Consume whitespace before the next input

Example from the lecture:

```
char a, b, c;

cout << "Enter at least 3 characters: ";
cin >> noskipws >> a >> ws >> b >> skipws >> c;

cout << a << b << c << endl;
```

Think of the input as a sequence of characters. These manipulators control whether whitespace is treated as data or skipped.

---

5. File Streams

Include:

```
#include <fstream>
```

A file can be:

- a **source** for an input stream
- a **destination** for an output stream

`ifstream` — Read From a File

```
ifstream ifile;
ifile.open("MyFile.txt");
```

Concept:

```
MyFile.txt -> ifile -> Program
```

`ofstream` — Write to a File

```
ofstream ofile;
ofile.open("Output.txt");
```

Concept:

```
Program -> ofile -> Output.txt
```

Typical File Workflow

1. Create a stream object.
2. Open the file.
3. Read/write data.
4. Close the file.

```
ifstream ifile;
ofstream ofile;

ifile.open("MyFile.txt");
ofile.open("Output.txt");

// read/write here

ifile.close();
ofile.close();
```

Opening associates the stream object in the program with the external file. Closing breaks that association.

---

6. File Opening Modes

The lecture example uses:

```
ofile.open("Output.txt", ios::out | ios::app);
```

- `ios::out` — output mode
- `ios::app` — append new output to the end of the file
- `|` combines the modes

---

7. Reading From Files

A. Read Formatted Data with `>>`

```
int x;
ifile >> x;
```

This behaves similarly to:

```
cin >> x;
```

The main difference is the source:

```
cin   -> keyboard
ifile -> file
```

B. Read One Character at a Time with `get()`

Version 1

```
int ch;

while ((ch = ifile.get()) != EOF) {
    cout.put(ch);
}
```

`get()` returns the next character, and `EOF` indicates end of file.

Version 2

```
char ch;

while (ifile.get(ch)) {
    cout.put(ch);
}
```

Here, the stream can be used as a condition. When reading fails/end-of-file is reached, the condition becomes false.

C. Read One Line at a Time with `getline()`

```
string line;

while (getline(ifile, line)) {
    cout << line << endl;
}
```

`getline()`:

- reads the next line
- stores it in a `string`
- discards the end-of-line character
- becomes false when it cannot read another line

Example:

```
int max = 0;
string line;

while (getline(ifile, line)) {
    if (line.length() > max)
        max = line.length();
}
```

This finds the length of the longest line.

---

8. Useful Stream State Operations

Operation	Meaning
`stream.fail()`	True if the stream is in a failure state
`stream.eof()`	True when the stream has reached end-of-file after a read attempt
`stream.clear()`	Reset stream status
`if (stream)`	Check whether the stream is currently valid
`stream.open(filename)`	Open/attach a file
`stream.close()`	Close/detach a file

A common and clean pattern is to let the read operation itself control the loop:

```
while (getline(ifile, line)) {
    // use line
}
```

or:

```
while (ifile.get(ch)) {
    // use ch
}
```

---

9. Text Files vs. Strings

Both contain character data, but they are different.

Text File	String
Stored externally	Lives inside the program
Persistent until deleted	Exists only while its variable/object exists
Can be exposed to multiple programs	Belongs to a running program
Commonly accessed sequentially	Can be manipulated directly in memory
File streams use `<fstream>`	String streams use `<sstream>`

Files are usually read from beginning to end sequentially.

---

10. String Streams

Include:

```
#include <sstream>
```

A **string stream** lets a string behave like the source/destination of a stream.

`istringstream` — String -> Program

```
string line = "Ann F 12345";

istringstream iss(line);

string name;
char sex;
long ID;
```

```
iss >> name >> sex >> ID;
```

Concept:

```
"Ann F 12345"
      |
      v
     iss
      |
      +--> name = "Ann"
      +--> sex  = 'F'
      +--> ID   = 12345
```

`istringstream` is similar to `ifstream`, except:

```
ifstream      : File   -> Program
istringstream : String -> Program
```

`ostringstream` — Program -> String

```
int i = 10;

ostringstream oss;
oss << "My favorite number is: " << i;

string s = oss.str();
```

Concept:

```
Program -> oss -> String
```

Use:

```
oss.str()
```

to obtain the resulting string.

`ostringstream` is similar to `ofstream`, except:

```
ofstream      : Program -> File
ostringstream : Program -> String
```

`stringstream`

The lecture also demonstrates:

```
stringstream ss;

ss << "My favorite number is: " << i;

string s = ss.str();
```

It allows stream-style manipulation of string data.

---

11. Big Picture

The most important idea in this lecture is that the syntax stays very similar even when the source/destination changes.

```
INPUT

Keyboard -> cin           -> Variable
File     -> ifstream      -> Variable
String   -> istringstream -> Variable


OUTPUT

Variable -> cout          -> Console
Variable -> ofstream      -> File
Variable -> ostringstream -> String
```

So these:

```
cin >> x;
ifile >> x;
iss >> x;
```

all mean essentially:

> Read data from an **input stream** into `x`.

And these:

```
cout << x;
ofile << x;
oss << x;
```

all mean essentially:

> Send `x` into an **output stream**.

---

12. Full Lecture-Style Example

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

using namespace std;

int main() {
    ifstream ifile;
    ofstream ofile;

    string inputFile;
    string outputFile;
    string line;

    string name;
    char sex;
    long ID;

    cout << "Enter the input and output file names:" << endl;
    cin >> skipws >> inputFile >> outputFile;
```

```
    ifile.open(inputFile.c_str());
    ofile.open(outputFile.c_str(), ios::out | ios::app);

    while (getline(ifile, line)) {

        istringstream iss(line);
        ostringstream oss;

        iss >> skipws >> name >> sex >> ID;

        oss << name << "\t"
            << sex << "\t"
            << ID << endl;

        ofile << oss.str();
        cout << oss.str();
    }

    ifile.close();
    ofile.close();

    return 0;
}
```

Flow:

```
Input File
    |
    v
 ifstream
    |
 getline()
    |
    v
  string line
    |
    v
istringstream
    |
    +--> name
    +--> sex
    +--> ID
    |
    v
ostringstream
    |
    +------> cout ------> Console
    |
    +------> ofile -----> Output File
```

Quick Review

```
cin >> x;                  // keyboard -> x
cout << x;                 // x -> console

ifstream ifile;            // input file stream
ofstream ofile;            // output file stream

getline(ifile, line);      // read one line
ifile.get(ch);             // read one character

istringstream iss(line);   // string -> input stream
```

```
ostringstream oss;          // output stream -> string

iss >> x;                   // extract data from string
oss << x;                   // insert data into string stream
oss.str();                  // get resulting string
```

One sentence to remember

> **Stream = an object used to manipulate the flow of data; only the source or destination changes.**